Instituto Politécnico Nacional
Centro de Investigación en Computación

Clasificación Inteligente de Patrones (A25)
Prof.: Dr. Antonio Alarcón Paredes

Tarea 4

Luis Angel Rojo Chavez
arojocz@gmail.com
lrojoc2024@cic.ipn.mx

19 de marzo de 2025

K neareast neighbours

1. Código en Python

```
        import pandas as pd
import numpy as np
from sklearn.model_selection import KFold
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import LeaveOneOut

def euclidean_distance(x1, x2):
    return np.sqrt(np.sum((x1 - x2) ** 2))

def knn(train_data, train_labels, test_point, k):
    distances = []

    # Calcular distancias desde el punto de prueba a todos los puntos de entrenamiento
    for i, row in train_data.iterrows():
        dist = euclidean_distance(row.values, test_point)
        distances.append((dist, train_labels[i]))

    #tuple with order of distance (distancia, etiqueta)
    distances.sort(key=lambda x: x[0])
    k_neighbors = [label for _, label in distances[:k]]

    class_counts = {}
    for label in k_neighbors:
        if label in class_counts:
            class_counts[label] += 1
        else:
            class_counts[label] = 1

    # best class
    most_common = max(class_counts, key=class_counts.get)
```

```python
    return most_common, class_counts

def k_fold_cross_validation(df, k_folds, k_neighbors):
    X = df[df.columns[:-1]]  # features
    y = df.iloc[:, -1]  # class

    kf = LeaveOneOut() if k_folds == df.shape[0] else KFold(n_splits=k_folds,
    shuffle=True, random_state=4)
    all_TP = all_TN = all_FP = all_FN = 0

    # Para cada fold, definimos particiones de entrenamiento y prueba
    for train_index, test_index in kf.split(X):
        X_train, X_test = X.iloc[train_index], X.iloc[test_index]
        y_train, y_test = y.iloc[train_index], y.iloc[test_index]
        y_pred = []
        for i, test_point in X_test.iterrows():
            predicted_class, class_counts = knn(X_train, y_train,
            test_point.values, k=k_neighbors)
            y_pred.append(predicted_class)

        cm = confusion_matrix(y_test, y_pred, labels=list(y.unique().tolist()))
        # Para cada clase, calculamos TP, TN, FP, FN
        for i in range(len(cm)):
            if len(cm) == 2:
                TP = cm[0, 0]
                FP = cm[1, 0]
                FN = cm[0, 1]
                TN = cm[1, 1]
                all_TP += TP
                all_TN += TN
                all_FP += FP
                all_FN += FN
                break
            else:
                TP = cm[i, i]
                FP = sum(cm[:, i]) - TP
                FN = sum(cm[i, :]) - TP
                TN = np.sum(cm) - (TP + FN + FP)

            all_TP += TP
            all_TN += TN
            all_FP += FP
            all_FN += FN

    #métricas de evaluación
```

```python
    recall = all_TP / (all_TP + all_FN)
    specificity = all_TN / (all_TN + all_FP)
    balance_acc = (recall + specificity) / 2
    valid_metrics = [recall, specificity, balance_acc]

    return valid_metrics, all_TP, all_TN, all_FP, all_FN, k_folds, k_neighbors

data_names = ['iris.data', 'led7digit.dat', 'heart.dat']


for data_name in data_names:
    df = pd.read_csv(data_name, comment='@', header=None, delimiter=',')

    for i in range(2):
        if i == 0:
            k_folds = 5
        if i == 1:
            leng = len(df)
            k_folds = leng

        valid_metrics, all_TP, all_TN, all_FP, all_FN, k_folds,
        k_neighbors= k_fold_cross_validation(df, k_folds, k_neighbors=5)

        # Mostrar resultados
        print(f"Archivo: {data_name}, k-folds: {k_folds}, k: {k_neighbors}")
        print(f"TP: {all_TP}")
        print(f"TN: {all_TN}")
        print(f"FP: {all_FP}")
        print(f"FN: {all_FN}")
        print(f"Recall: {valid_metrics[0]}")
        print(f"Specificity: {valid_metrics[1]}")
        print(f"Balanced Accuracy: {valid_metrics[2]}")
        print('---' * 15)


```

Parte A

Para la realización de este programa, se usan las herramientas de `scikit learn` como lo es obtener el reporte para las métricas de validación, matriz de confusión y demás. Se usan 5 folds en el método de validación cruzada y como es el algoritmo de knn se estable 5 vecinos como valor de k.

Iris

- **k-fold cross-validation**: usando 5 divisiones

  Aunque no es necesario y en clase establecimos un programa usando report de scikit learn, como reporte y anécdota se usó una ponderación clase por clase para los valores de la matriz de confusión y luego obteniendo un promedio en las métricas como recall, specificity y balanced accuracy, a continuación se desarrolla la idea en fórmulas para que quede más claro.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{\sum\limits_{\forall c_i \in C} \text{TP}_{c_i}}{\sum\limits_{\forall c_i \in C} \text{TP}_{c_i} + \sum\limits_{\forall c_i \in C} \text{FN}_{c_i}}$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}} = \frac{\sum\limits_{\forall c_i \in C} \text{TN}_{c_i}}{\sum\limits_{\forall c_i \in C} \text{TN}_{c_i} + \sum\limits_{\forall c_i \in C} \text{FP}_{c_i}}$$

$$\text{Balanced Accuracy} = \frac{\text{Recall} + \text{Specificity}}{2}$$

```
Archivo: iris.data, k-folds: 5, k: 5
TP: 145
TN: 295
FP: 5
FN: 5
Recall: 0.9666666666666667
Specificity: 0.9833333333333333
Balanced Accuracy: 0.975
--------------------------------------------
```

Figura 1: Resultados de iris para 5 folds.

```
Archivo: led7digit.dat, k-folds: 5, k: 5
TP: 347
TN: 4347
FP: 153
FN: 153
Recall: 0.694
Specificity: 0.966
Balanced Accuracy: 0.83
--------------------------------------------
```

Figura 2: Resultados para led7digit con 5 folds.

```
Archivo: heart.dat, k-folds: 5, k: 5
TP: 65
TN: 107
FP: 43
FN: 55
Recall: 0.5416666666666666
Specificity: 0.7133333333333334
Balanced Accuracy: 0.6275
---------------------------------------------
```

Figura 3: Resultados para heart con 5 folds

- **leave-one-out**

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

$$\text{Balanced Accuracy} = \frac{\text{Recall} + \text{Specificity}}{2}$$

```
Archivo: iris.data, k-folds: 150, k: 5
TP: 145
TN: 295
FP: 5
FN: 5
Recall: 0.9666666666666667
Specificity: 0.9833333333333333
Balanced Accuracy: 0.975
---------------------------------------------
```

Figura 4: Resultados para iris con leave one out.

```
Archivo: led7digit.dat, k-folds: 500, k: 5
TP: 355
TN: 4355
FP: 145
FN: 145
Recall: 0.71
Specificity: 0.9677777777777777
Balanced Accuracy: 0.8388888888888888
---------------------------------------------
```

Figura 5: Resultados para led7digit con leave one out.

```
Archivo: heart.dat, k-folds: 270, k: 5
TP: 67
TN: 117
FP: 33
FN: 53
Recall: 0.5583333333333333
Specificity: 0.78
Balanced Accuracy: 0.6691666666666667
----------------------------------------
```

Figura 6: Resultados para heart con leave one out.

Parte B

- ¿Por qué cuando $k = 1$ clasifica correctamente el 100 % de patrones y cuando $k \neq 1$ no?
  Cuando el dataset es el mismo el de entrenamiento y prueba, la distancia consigo mismo es 0, entonces se asigna la clase de si mismo, (como hacer trampa).
- ¿Qué pasa con la frontera de decisión cuando k es muy pequeño?
  La frontera se hace muy irregular.
- ¿Qué pasa con la frontera de decisión cuando k es muy grande?
  Una frontera de decisión que toma la decisión basada en muchos votos es más suave e informada.
- ¿Qué pasa con la frontera de decisión y el comportamiento del clasificador cuando k es un número mayor que la suma de patrones en dos clases (iris) o mayor que la cantidad de patrones en una clase (clasificación biclase)? Puede eliminar una clase.